Magnitude and Phase Aware Spectral Convolutional Neural Networks: Explainable Frequency-Domain Classification

Anonymous CVPR submission

Paper ID *****

Abstract

Convolutional neural networks (CNNs) have achieved strong performance in visual recognition; however, standard architectures are typically designed for spatial-domain inputs. In contrast, this work uses frequency-domain inputs obtained via the fast Fourier transform (FFT), which can expose discriminative high-frequency patterns that are less apparent in the spatial domain. Despite this advantage, interpretability is challenging in the spectral domain because it is non-trivial to identify where the model focuses in the original image space, especially in medical imaging, where transparent decision support is essential. A spectral learning framework with explainability is presented for frequency-domain CNN inference. Each red, green, and blue (RGB) channel is transformed using the FFT to construct a 9-channel representation comprising magnitude, cosine phase, and sine phase. Phase components provide localisation-related cues, but they are also sensitive to perturbations and can introduce noise that degrades classification performance. To address this issue, first-order gradients of the activation maps are fused with the activation maps before global average pooling, thereby stabilising learning and recovering discriminative spectral signatures. For interpretability, a frequency-aware class activation mapping procedure is employed: salient spectral contributions are identified in the frequency domain and mapped back to the spatial domain to generate class-discriminative attribution maps. The resulting explanations provide improved localisation with reduced visual artefacts, supporting more reliable interpretation for medical imaging applications.

1. Introduction

Convolutional neural networks (CNNs) have transformed the field of visual recognition, demonstrating strong performance across a wide range of image classification tasks[2]. Architectures such as ResNet-50 are conventionally designed to operate on spatial-domain inputs, where pixel intensities are processed directly[21]. However, the spatial domain does not always surface all discriminative information present in an image[16]. Frequency-domain representations, obtained through the fast Fourier transform (FFT), can reveal high-frequency structural patterns that are less apparent to the naked eye and often underutilised by standard spatial CNN pipelines[4]. This distinction becomes particularly valuable in medical imaging, where subtle textural and structural cues can be clinically significant.

To leverage these advantages, this work reformulates image input from the spatial domain to the frequency domain[13]. Each of the three red, green, and blue (RGB) colour channels is independently transformed using the FFT, yielding three components per channel: magnitude, cosine phase, and sine phase. These components are concatenated to form a 9-channel frequency-domain representation, which serves as the input to the CNN[13][16]. This design enables the model to learn discriminative spectral features from its very first layer, rather than implicitly inferring frequency-related patterns from spatial activations[21]. However, while phase components carry localisation-related information, they are inherently sensitive to perturbations, and their inclusion can introduce noise that undermines classification performance[16]. To mitigate this, first-order gradients of the activation maps are fused with the activation maps before the global average pooling (GAP) step, effectively stabilising the learning process and reinforcing the recovery of discriminative spectral signatures[14].

Despite the representational benefits of frequency-domain learning, interpretability poses a non-trivial challenge[16][21]. In the spectral domain, it is not straightforward to identify which regions of the original image the model is attending to, since frequency components do not have a direct spatial correspondence. This limitation is especially consequential in medical imaging applications, where transparent and trustworthy decision support is not merely desirable but essential[5][7][11]. To address this, the framework incorporates a frequency-aware class

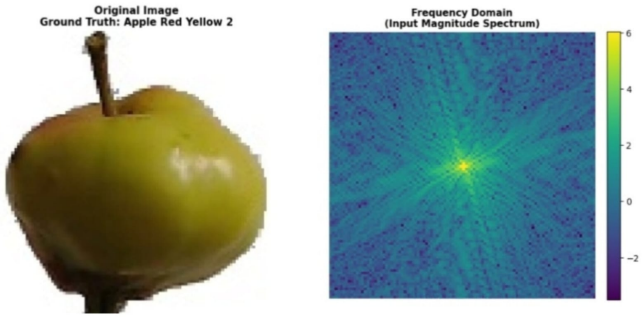


Figure 1. **Illustration of spatial and frequency domain representations of an image**. The left panel shows the spatial domain image, while the right panel shows its corresponding frequency-domain representation obtained by using FFT

activation mapping procedure. Salient spectral contributions are first identified in the frequency domain and subsequently mapped back to the spatial domain, producing class-discriminative attribution maps that localise diagnostically relevant regions with reduced visual artefacts[14].

Taken together, this paper presents a spectral learning framework that unifies frequency-domain inference, gradient-stabilised feature fusion, and spatially interpretable explanations. The resulting system not only improves classification performance over standard spatial baselines but also produces more reliable and interpretable outputs, making it well-suited for deployment in safety-critical domains such as medical image analysis.

2. Related Work

The spatial domain refers to the conventional representation of images where pixel values are arranged in their original two-dimensional layout, directly corresponding to physical locations—this is the standard input format for most CNN architectures. However, spatial-domain approaches have notable disadvantages: discriminative high-frequency patterns (like subtle edges or fine textures) are often less apparent in this representation, limiting both classification accuracy and the model's ability to capture nuanced features critical for tasks like medical imaging.

The frequency domain represents images in terms of magnitude and phase components, exposing high-frequency patterns that remain hidden in spatial representations. The frequency domain offers advantages by revealing discriminative spectral signatures that enhance classification performance, while phase components inherently encode spatial localisation cues.

Numerous researchers have extensively explored the frequency domain and made substantial contributions. Nahid and the team advanced histopathological breast cancer image classification by integrating frequency-domain features (DFT, DCT) with a residual-block-based CNN. They also proposed CNN-DF and CNN-DC to systematically study how CNNs learn and interpret global spectral information using exclusively frequency-domain inputs [10]. Ee Fey Goh and team proposed a frequency domain CNN (FD-CNN) for high-resolution diabetic retinopathy images, introducing RFFT-based FDC and FDP layers and a channel-independent convolution (CIC) strategy to reduce computational complexity and memory usage while preserving fine-grained clinical details[4]. Kai Xu and team addressed accuracy loss from spatial downscaling by reshaping high-resolution images in the frequency domain using DCT coefficients and feeding them into minimally modified CNNs. They further introduced a learning-based dynamic channel selection strategy, revealing CNNs' spectral bias toward low frequencies[21]. Totterstrom investigated the direct use of Fourier magnitude and phase spectra as inputs to an AlexNet-based CNN and demonstrated that standard CNNs perform worse on purely frequency-domain images than on spatial (RGB) inputs[16]. Xianlun Tang and team proposed tKFC-Net, a U-Net-based segmentation model that integrates FFT-based frequency representations into convolution, pooling, and upsampling. Their twin-kernel Fourier convolution (t-KFC) layer separates high- and low-frequency components to capture semantic context and fine boundary details across medical imaging datasets[15].

Across multiple studies, integrating frequency-domain representations into deep learning models consistently outperforms purely spatial CNNs, particularly in high-resolution medical imaging. For example, breast cancer classification improved from 84.43 to 87.47% - 93.00 to 97.19%, diabetic retinopathy detection increased from 92.49% to 95.63%, and tKFC-Net achieved higher F1/Dice scores than U-Net baselines [4][10][15]. These gains arise from preserving fine-grained details without aggressive downsampling, explicitly separating low- and high-frequency components for targeted feature extraction, and leveraging global spectral representations to mitigate spectral bias and reduce redundancy.

Recent advancements in integrating frequency-domain transformations reveal that the primary focus of these studies has been overwhelmingly directed toward improving computational efficiency and classification or segmentation accuracy. However, despite these performance gains, a critical gap remains: none of the existing literature demonstrates instance-level explainability within the frequency domain.

'Explainability/XAI' refers to the techniques and approaches used to analyse and interpret how these complex models arrive at their decisions. XAI aims to provide an understanding of how DL models reach their predictions or classifications [5]. The generally used XAI techniques are LIME and SHAP.

LIME (Local Interpretable Model-agnostic Explana-

tions) is a post-hoc explanation technique that interprets individual predictions of any black-box model by approximating it with a simple, interpretable model (such as a linear model) in the local neighbourhood of a specific instance[12]. For images, LIME segments the input into superpixels, generates perturbed samples by randomly masking them, weights these samples based on similarity to the original image, and fits a simple linear model to estimate each superpixel's contribution to the prediction. [5][12]

SHAP (SHapley Additive exPlanations) is a model-agnostic explanation framework that assigns each feature an importance value for a specific prediction using Shapley values from game theory, representing how each feature contributes to the change in the expected model output[6]. For images, it estimates the contribution of pixels or pixel groups to the prediction. It satisfies three key properties: local accuracy, missingness, and consistency. It computes feature attributions (the process of assigning an importance value to each input feature to quantify how much it contributed to a specific model prediction) using methods such as Kernel SHAP (weighted linear regression) or Deep SHAP (for neural networks). [5][6]

Even though LIME and SHAP are good, they have significant disadvantages that make them less useful for image explainability. Lack of precise spatial localisation: LIME explains predictions with super-pixel masks that can vary across runs and do not pinpoint exact boundaries [5]. Computational Expense: Generating a LIME explanation for a deep network such as inception can take around 10 minutes per image, precluding real-time use [12]. Computational Intractability for High-Dimensional Images: Exact SHAP requires enumerating 2^M subsets (NP-hard with $O(2^M+M^3)$ complexity), which is impossible for typical $224 \times 224 \times 3$ images [6]. Feature independence assumption: Model-agnostic SHAP approximations treat pixels as independent when estimating conditional expectations, producing unrealistic baseline samples (e.g., a tumour pixel surrounded by "missing" tissue) that compromise the reliability of the attributions [5][6].

To overcome the disadvantages in LIME and SHAP, we use a more efficient XAI technique named CAM (Class Activation Map) to interpret the explainability in images more effectively, as it provides high-resolution and precise spatial localisation by leveraging the spatial structure of convolutional feature maps to accurately highlight complete objects and fine-grained details [14] and drastically reduces computational expense by requiring only a single forward and partial backward pass instead of heavy perturbation sampling [14]. It even offers broad applicability without architectural modifications [1].

Even though CAM is effective compared to LIME and SHAP, there are a few disadvantages of CAM which make it difficult to operate in the frequency domain, like loss of high-frequency information, noisy and unstable explanations, inability to provide frequency-specific explanations, and architectural constraints and computational overhead.

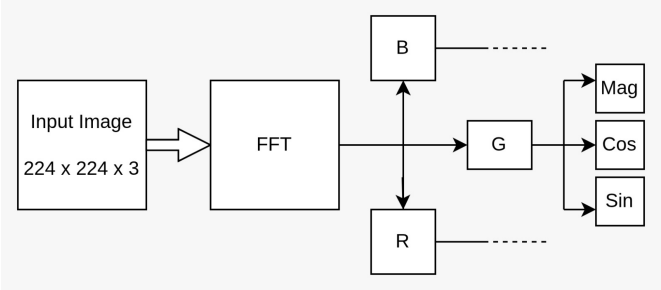


Figure 2. Each RGB channel is decomposed in the frequency domain into magnitude, cosine-phase, and sine-phase components (illustrated for the G channel), forming a nine-channel spectral representation that serves as input to the CNN for improved feature robustness and localisation.

Despite extensive research on integrating frequency-domain representations into deep learning models, ranging from classification and segmentation to computational acceleration, prior work has focused almost exclusively on improving performance and efficiency in the frequency domain. Even though these approaches have demonstrated clear advantages over purely spatial-domain methods, none have introduced the concept of explainability in the frequency domain. This paper primarily focuses on introducing explainability in the frequency domain using CAM, enabling the visualisation of which frequencies have contributed to the prediction and mapping these back to the spatial domain to reveal which pixel intensities have influenced the model's decision.

3. Methodology

3.1. Spatial-to-frequency domain conversion

An RGB image, which has three colour channels, is transformed into the frequency domain. This is done by applying the 2D discrete Fourier transform separately to each colour channel, resulting in a complex spectrum. The spectrum is then adjusted so that the lower frequency parts are in the middle using a method called 'fftshift'. Once this is done, the spectrum is split into two parts: magnitude and phase. The magnitude, $M_c(u, v)$, shows how much of each frequency is present, which relates to texture and edges in the image. The phase, $\Phi_c(u, v)$, keeps the spatial arrangement of the image intact.

$$F(u,v) = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) e^{-j2\pi(ux/M+vy/N)} \quad (1)$$

where u and v represent the spatial frequency coor-

dinates [13]. The resulting output $F(u, v)$ is complex-valued, consisting of a real part (Re) and an imaginary part (Im).[2][13]

$$M_c(u, v) = |F(k, l)| = \sqrt{Re(F(k, l))^2 + Im(F(k, l))^2} \tag{2}$$

Here, (k, l) are the frequency positions. [16]

$$\Phi_c(u, v) = \arg(F(k, l)) = \arctan\left(\frac{Im(F(k, l))}{Re(F(k, l))}\right) \tag{3}$$

To deal with the wide range of values in the magnitude spectrum, a logarithmic compression method is used along with some numerical adjustments. This is done using:

$$L_c(u, v) = \ln(\max(M_c(u, v), \epsilon) + \epsilon) \tag{4}$$

This helps make the data more consistent for learning. Since phase values repeat and can have sudden changes, they are converted using sine and cosine functions:

$$\begin{aligned} P_c^{\cos}(u, v) &= \cos(\Phi_c(u, v)), \\ P_c^{\sin}(u, v) &= \sin(\Phi_c(u, v)) \end{aligned} \tag{5}$$

This creates a smooth and controlled representation. The processed log-magnitude and phase data from three different channels are then combined into a single nine-channel frequency tensor

$$\mathbf{X}_{\text{freq}} \in \mathbb{R}^{9\times H\times W}$$

. This format keeps both the energy distribution and the spatial details, making it suitable for further processing with convolutional methods.

3.2. CNN architecture

The architecture used is a modified version of a CNN model that works with 9-channel frequency-domain inputs. The original CNN model is designed for three-channel RGB images, so the first convolutional layer is changed such that it takes 9 channels and produces l channels, with half the height and width of the input. It uses a 7x7 kernel, a stride of 2, and padding of 3.

$$\text{Conv1} : \mathbb{R}^{9\times H\times W} \to \mathbb{R}^{l\times \frac{H}{2}\times \frac{W}{2}}$$

To use the pre-trained weights from ImageNet, a transfer-and-scale method is applied. The first three channels of the new layer are set to the pre-trained weights, while the remaining six channels are initialised using a scaled version of the Kaiming He initialisation. This setup keeps the useful spatial features from the pre-trained model while allowing the model to learn frequency-domain information gradually.

$$W_{\text{conv1}}[:, 0{:}3, :, :] = W_{\text{pretrained}}, \tag{6}$$

$$W_{\text{conv1}}[:, 3{:}9, :, :] = 0.1 \cdot W_{\text{Kaiming}} \tag{7}$$

After the modified input layer, features are taken out using the standard CNN residual blocks (layers 1 to 4). Each block does the following:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, W_i) + \mathbf{x} \tag{8}$$

which $\mathcal{F}$ includes steps like convolution, batch normalisation, and applying the ReLU function. The last layer (layer 4) gives out deep feature maps:

$$\mathbf{A} \in \mathbb{R}^{B\times C_{\text{backbone}}\times h\times w}$$

where B is the batch size, C_{backbone} is the number of feature channels produced from the last convolutional layer, and $h \times w$ denotes the spatial resolution of the feature maps. These feature maps are used in the next steps for combining information from two different domains and for making the model's decisions easier to understand.

3.3. Dual-frequency feature fusion

The dual-frequency feature fusion module works with activation maps $\mathbf{A} \in \mathbb{R}^{B\times C\times h\times w}$ that are taken from the last residual block. To get more information about the spectral details in these spatial features, a second-level 2D FFT is done separately for each channel.

$$G_k(u, v) = \text{FFT2D}(A_k(x, y)), \quad M_k^{(2)} = |G_k|. \tag{9}$$

This gives magnitude spectra, which are then log-scaled and normalised for each channel to create frequency-domain feature maps $\mathbf{A}_{\text{freq}}$. This process helps reveal patterns like repeating structures, main spatial scales, and periodic elements that aren't easily seen in the original spatial data, making the representation richer with more complex structural details.

Because spatial and frequency features exist in different numerical ranges, separate batch normalisation layers are used to match their distributions

$$\tilde{\mathbf{A}} = \text{BN}_{\text{spatial}}(\mathbf{A}), \quad \tilde{\mathbf{A}}_{\text{freq}} = \text{BN}_{\text{freq}}(\mathbf{A}_{\text{freq}}) \tag{10}$$

$$\mathbf{C} = [\tilde{\mathbf{A}};\ \tilde{\mathbf{A}}_{\text{freq}}] \in \mathbb{R}^{B\times 2C\times h\times w} \tag{11}$$

The normalised features are combined along the channel dimension and then go through a channel attention mechanism that uses global average pooling.

$$\mathbf{z} = \text{GAP}([\tilde{\mathbf{A}}; \tilde{\mathbf{A}}_{\text{freq}}]), \quad \mathbf{s} = \sigma(W_2, \delta(W_1\mathbf{z})). \tag{12}$$

where GAP denotes global average pooling, δ is the ReLU activation function, σ is the sigmoid activation function, $W_1 \in \mathbb{R}^{C\times 2C}$ is the dimensionality reduction matrix, and $W_2 \in \mathbb{R}^{2C\times C}$ is the expansion matrix.

$$\hat{\mathbf{C}} = \mathbf{s} \odot \mathbf{C} \tag{13}$$

This attention process helps the model focus on important spatial and frequency channels while reducing the impact of less useful ones. It learns how different channels relate to each other across both domains.

Finally, the weighted features are split and merged using a learnable fusion method:

$$\hat{\mathbf{A}}_s = \hat{\mathbf{C}}[:, :C, :, :], \quad \hat{\mathbf{A}}_f = \hat{\mathbf{C}}[:, C:, :, :]$$

$$\mathbf{A}_{\text{fused}} = \alpha \cdot \hat{\mathbf{A}}_s + \beta \cdot \hat{\mathbf{A}}_f \tag{14}$$

where $\alpha = \sigma(\alpha \cdot \text{raw})$ and $\beta = \sigma(\beta \cdot \text{raw})$ are trainable values that stay between 0 and 1. These values start by giving more importance to spatial features because they have better performance in the beginning. As the model learns, it changes how much weight it gives to frequency information. The final combined feature includes both local details and overall spectral patterns before going through average pooling and classification.

The combined feature maps $\mathbf{A}_{\text{fused}} \in \mathbb{R}^{B\times C_{\text{backbone}}\times h\times w}$ are first turned into a fixed-length form using global average pooling. This process creates two vectors:

$$\mathbf{v} = \frac{1}{h\cdot w}\sum_{i=1}^{h}\sum_{j=1}^{w}, \quad \mathbf{A}_{\text{fused}}(:,:,i,j) \in \mathbb{R}^{B\times C_{\text{backbone}}} \tag{15}$$

which gather spatial information while making the model less sensitive to changes in space.

This pooled vector v is then sent through a three-layer fully connected classification part as shown.

$$\mathbf{v} \xrightarrow{\text{Dropout}(p_1)} \xrightarrow{W_1\in\mathbb{R}^{C_{\text{backbone}}\times C_1}} \xrightarrow{\text{ReLU}} \xrightarrow{\text{BN}} \xrightarrow{\text{Dropout}(p_2)} \xrightarrow{W_2\in\mathbb{R}^{C_1\times C_2}} \xrightarrow{\text{ReLU}} \xrightarrow{\text{BN}} \xrightarrow{\text{Dropout}(p_3)} \xrightarrow{W_3\in\mathbb{R}^{C_2\times K}} \mathbf{o}$$

The output is a vector $\mathbf{o} \in \mathbb{R}^{B\times K}$ that gives the predicted scores for K classes. Using ReLU activation along with batch normalisation helps the model learn better and train faster. The dropout rates start at p_1, then go down to p_2, and finally to p_3. This setup applies more regularisation in the early layers and less in the later ones, helping to prevent overfitting while still allowing the model to capture enough details for accurate predictions.

3.4. Explainability

Class activation mapping (CAM) variants include Grad-CAM, Grad-CAM++, XGrad-CAM, Smooth Grad-CAM, Score-CAM, Ablation-CAM, and Eigen-CAM. In this subsection, Score-CAM is implemented in the frequency domain; the same procedure can be applied to other CAM-based methods. Score-CAM works on the combined activation maps $\mathbf{A}_{\text{fused}} \in \mathbb{R}^{1\times C\times h\times w}$, which are created before the global pooling step. For each input, each channel is scaled up to the original image size using a bilinear method. Then, these channels are adjusted to fit a range from 0 to 1. This makes them suitable as soft importance maps that show where the model focuses its attention in space.

$$\bar{A}_k = \frac{A_k^{\uparrow} - \min(A_k^{\uparrow})}{\max(A_k^{\uparrow}) - \min(A_k^{\uparrow})} \tag{16}$$

where it $A_k^{\uparrow}$ is called an unsampled activation map. The normalised map, $\bar{A}_k$, acts as a soft spatial mask. This mask is applied to the frequency-domain input, resulting in $\mathbf{X} * k = \mathbf{X} * \text{freq} \odot \bar{A}_k$. This process helps keep the areas that are most important according to the (k)-th fused feature.

Each of these masked inputs goes through the whole network to produce a score that shows how confident the model is about each class.

$$s_k = \text{softmax}((f(\mathbf{X}_k)) * t) \tag{17}$$

where $f(\cdot)$ denotes the model prediction function and t is the target class index.

Finally, the class activation map is created by combining the original fused feature maps, using the confidence scores as weights, and then applying ReLU activation to keep only the areas that contribute positively.

$$\text{CAM}(u,v) = \text{ReLU}\left(\sum_{k=1}^{C} s_k \cdot A_k(u,v)\right) \tag{18}$$

This keeps the original Score-CAM idea and makes sure the explanations are clearly connected to the spatial and frequency patterns that the DFFF learns together.

3.5. Frequency to spatial domain mapping

The CAM created in the frequency domain is converted back to the spatial domain so people can easily understand it. The CAM acts as a mask that multiplies the frequency magnitude. When combined with the original phase, the inverse FFT creates a saliency signal in the spatial domain.

$$I_{\text{sal}} = \text{IFFT2D}\big(\text{CAM} \cdot M \cdot e^{j\Phi}\big) \tag{19}$$

The final saliency map is made by averaging across all channels. It is then flipped because areas with high energy

Algorithm 1 Dual-frequency feature fusion with Second-level FFT and Score-CAM

Input: Spatial image X
Output: Class prediction and dual-domain saliency map
Frequency domain transformation:
Apply FFT X to obtain magnitude and phase representations.
Frequency feature encoding:
Log-normalise magnitude and encode phase using sine and cosine components.
Frequency domain dataset construction:
Combine magnitude and phase features to form a multi-channel frequency input.
Forward pass through CNN backbone:
Pass frequency input through a modified ResNet to extract spatial feature maps.
Second-level FFT on feature maps:
Apply FFT to the last convolutional layer activations to capture hidden frequency patterns.
Dual-frequency feature fusion:
Fuse spatial and frequency features using learnable weights: $X_{fused} = \alpha X + \beta \text{FFT}(X)$
Channel attention refinement:
Adaptively reweight fused features using channel-wise attention.
Global average pooling:
Aggregate fused feature maps into a compact feature vector.
Classification head:
Predict the final class label using fully connected layers.
Explainability:
Generate class activation maps using fused feature responses without gradients.
Frequency-to-spatial mapping:
Apply inverse FFT using preserved phase to map CAMs back to the spatial domain.
Saliency refinement:
Smooth, threshold, and normalise the spatial saliency map.
Visualisation:
Overlay the refined saliency map on the original image for interpretability.

in the frequency domain are related to edges and textures, not the centres of objects.

Now, a Gaussian filter with a standard deviation of 2.5 is used to smooth the map with a threshold based on the $40th$ percentile. To make it cleaner, it goes through two steps: first, a closing operation using a 5x5 structuring element, and then an opening operation using a 3x3 structuring element. Finally, the contrast is increased using a power-law transformation with an exponent of 0.7.

The process to bring back spatial images from the frequency domain undoes the steps that were taken. With the log-magnitude representation L and a phase Φ, the complex spectrum can be reconstructed as

$$\hat{F}(u,v) = e^{\text{clamp}(L,-10,10)} \cdot e^{j\Phi(u,v)} \tag{20}$$

The spectrum is then shifted back using inverse fftshift, and the spatial image is created by applying the inverse FFT.

$$I_{\text{reconstructed}}(x,y) = \text{Re}\Big(\text{IFFT2D}\Big(\text{ifftshift}\Big(\hat{F}(u,v)\Big)\Big)\Big) \tag{21}$$

4. Experimentation and Results

4.1. Dataset used

This paper uses the Fruits-360 dataset authored by Mihai Oltean (https://www.kaggle.com/datasets/moltean/fruits), which is a big image classification dataset from Kaggle. It has images of fruits and vegetables that are 100×100 pixels in RGB format. Each image is taken against a white background using a motorised rotating system, so there are many different angles for each type. The dataset is split into two folders: one for training and one for testing.

4.2. Pre-processing steps

4.2.1. Training set

During training, input images go through a stochastic augmentation pipeline which includes random horizontal flipping with probability 0.5, random vertical flipping with probability 0.3, random rotation within ±20 degrees, colour jittering applied to brightness (±0.2), contrast (±0.2), saturation (±0.2), and hue (±0.1), and random affine transformations with up to 10% translation and scaling between 0.9 and 1.1. The pixel values are turned into tensors that range from 0 to 1, and then they are adjusted using specific average and standard deviation values from the ImageNet dataset: mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225].

4.2.2. Testing set

The test set undergoes only the deterministic steps: no data augmentation is applied, but it undergoes tensor conversion, ImageNet normalisation (the same as the training set) and FFT-based frequency domain conversion.

4.3. Training strategy

4.3.1. Loss function

The model is trained using cross-entropy loss with label smoothing. The loss function is:

Table 1. **Comparative performance of CNN backbones with the proposed dual-frequency feature fusion framework on the Fruits-360 test set.**

Metric	ResNet-50	MobileNetV2	DenseNet-121	EfficientNet-B0	VGG-19
Overall Accuracy (%)	90.39	89.94	89.65	87.78	86.51
Cohen's Kappa Score (%)	90.34	89.88	89.60	87.71	86.45
Overall Precision (%)	92.25	90.75	91.54	91.35	89.27
Overall Recall (%)	90.48	89.89	89.90	87.82	86.71
Overall F1 Score (%)	90.30	89.73	89.79	87.72	86.40
Overall Specificity (%)	99.96	99.94	99.95	99.93	99.94
Overall Error Rate (%)	9.52	0.10	0.08	16.18	14.48

$$\mathcal{L} = -\sum_{k=1}^{K} \tilde{y}_k \log(\hat{y}_k) \tag{22}$$

where the smoothed labels are defined as:

$$\tilde{y}_k = (1-\varepsilon)\, y_k + \frac{\varepsilon}{K}$$

$$\hat{\mathbf{y}} = \mathrm{softmax}(\mathbf{o})$$

where $\mathbf{y}$ one-hot encoded ground truth, $\hat{\mathbf{y}}$ is predicted probability. ε is a smoothing parameter with a value of 0.1. This prevents the model from becoming overconfident and improves generalisation.

4.3.2. Optimizer and learning rate

The AdamW optimiser is used with decoupled weight decay $\lambda = 5 \times 10^{-4}$. The learning rate approach used is the pre-trained CNN model, which is adjusted with a small learning rate $\eta_{\text{pre}} = 10^{-5}$, while the new parts added to the model, like the modified input convolution, the DFFF module, and the classification head, are trained with a larger learning rate $\eta_{\text{new}} = 5 \times 10^{-4}$. The learning rate follows a cosine annealing schedule with warm restarts:

$$\eta(t) = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{T_{\text{cur}}}{T_i}\pi\right)\right) \tag{23}$$

where $T_0 = 10$ is the initial restart period, $T_{\text{mult}} = 2$ the period multiplier, and $\eta_{\min} = 10^{-7}$. This schedule lets the optimiser break free from local minima by increasing the learning rate at regular intervals.

4.3.3. Mixed Precision Training

Training uses automatic mixed precision (AMP), where the main calculations happen with float16, but the updates to the model's settings are done with float32. The system automatically adjusts the scaling of gradients to make sure they stay accurate when using half-precision numbers.

4.3.4. Gradient clipping

To stop the gradients from getting too large, which can be a problem because the network uses FFT operations, we use a technique called global gradient norm clipping. This sets a maximum limit on how big the gradients can be, which is 0.5:

$$\mathbf{g} \leftarrow \frac{0.5}{\|\mathbf{g}\|_2} \cdot \mathbf{g} \quad \text{if } \|\mathbf{g}\|_2 > 0.5 \tag{24}$$

4.4. Setup

The code is built to work with a GPU and is best used on a 16 GB GPU that supports *CUDA*. It uses *PyTorch* as the main tool for deep learning and *torchvision* for using pre-trained models. *SciPy* is used for processing data after generating saliency maps, and *Matplotlib* is used to create visualisations. The code turns on CuDNN benchmark mode and turns off deterministic mode to make the training faster. When a CUDA device is available, it uses mixed-precision training through PyTorch's AMP with *GradScaler*, which helps use less memory and speeds up the training process.

The original training data is divided again into 85% for training and 15% for validation, using a fixed random seed of 42. The DataLoaders are set up with a batch size of 128, pin memory is turned on to make data transfers between the computer and device faster, and a prefetch factor of 2 is used when multiple data loading workers are involved.

4.5. Hyperparameters

The model was trained for 50 epochs using a batch size of 128 and started with a learning rate of 0.001. It used different learning rates for the layers that were already trained($1e^{-5}$), and the new layers added($5e^{-4}$). To prevent the model from overfitting on training data, suitable dropout rates were used in the classifier layers. Also, an early stopping method was used, which stopped training if

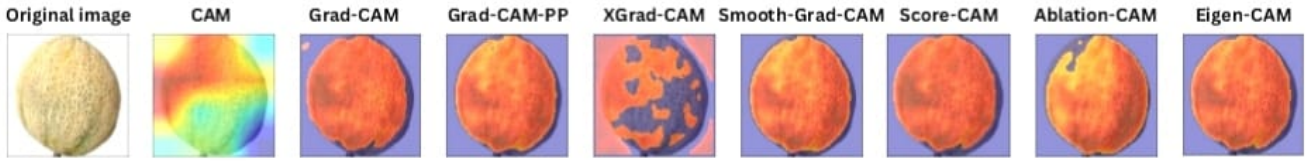


Figure 3. Qualitative comparison of frequency-domain heatmaps with class activation mapping methods.

the model's performance on validation data didn't improve for 15 epochs.

4.6. Quantitative results analysis

Table 1 shows the test results of the dual-frequency feature fusion method using five different CNN models on the Fruits-360 dataset, showing that adding frequency information at the start and during the feature extraction process works well. ResNet-50 achieved the best accuracy due to its deep residual design and skip connections that facilitate stable gradient flow through FFT-based transformations. MobileNetV2 attained 89.94% accuracy by efficiently handling the 9-channel frequency input through inverted residuals and linear bottlenecks. VGG-19 recorded the lowest accuracy because its older architecture lacks batch normalisation and residual connections, making frequency-domain training harder. DenseNet-121 achieved 89.65% accuracy, as dense feature reuse supports effective propagation of low-level spectral patterns. EfficientNet-B0 obtained 87.78% accuracy, as its compound scaling—originally optimised for spatial ImageNet images—may require further adaptation for frequency-domain inputs.

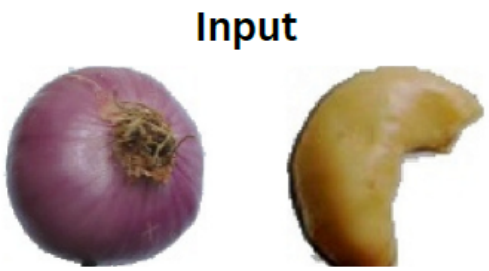
So, the proposed dual-frequency feature fusion framework effectively uses frequency information at both the input and feature levels, resulting in decent classification results.

4.7. Qualitative results analysis

Figure 3 illustrates the activation maps generated by eight different Class Activation Mapping (CAM) variants applied to a cantaloupe sample from the Fruits-360 dataset. Each variant offers distinct visualisation characteristics that reveal how the dual-frequency feature fusion model interprets frequency-enriched features during classification. Among all the variants tested, Score-CAM delivers the best overall visualisation for this framework because of its gradient-free, confidence-driven attribution mechanism, which makes it the most reliable and interpretable visualisation tool for the dual-frequency feature fusion framework.

5. Conclusion and Future Work

This paper proposes a nine-channel spectral representation in the frequency domain. Each RGB channel is decomposed into one magnitude component and two phase components



(a) Input image containing two fruit instances.

(b) Class activation map (Top-1 prediction).

Figure 4. Qualitative visualisation of frequency-aware localisation. The activation map highlights discriminative regions corresponding to the top-1 predicted class, demonstrating spatial focus in the frequency domain.

(sine and cosine). The sine–cosine formulation improves localisation robustness by preserving continuous phase information. However, explicit phase encoding may introduce noise that degrades performance. To mitigate this, second-order gradients from the final convolutional feature maps are fused with the features using an attention-based dual–frequency fusion module. Training stability is maintained through NaN/Inf safeguards, mixed-precision training, and gradient clipping. CAM methods evaluated on a fruit dataset highlight frequency-informed regions, showing that spectral cues complement spatial features with minimal computational overhead.

Future work will introduce a multi-resolution frequency pyramid (100×100, 50×50, 25×25) with lightweight CNN processing and attention-based fusion. A multi-scale CAM framework will further provide frequency-aware interpretability.

References

[1] Aditya Chattopadhay, Anirban Sarkar, Prantik Howlader, and Vineeth N Balasubramanian. Grad-cam++: Generalized gradient-based visual explanations for deep convolutional networks. In *IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 839–847. IEEE, 2018. 3

[2] Wenlin Chen, James Wilson, Stephen Tyree, Kilian Q Weinberger, and Yixin Chen. Compressing neural networks with the hashing trick. In *International Conference on Machine Learning (ICML)*, pages 2285–2294. PMLR, 2016. 1, 4

[3] Rui Fu, Qingyong Hu, Xiaohu Dong, Yulan Guo, Ying Gao, and Biao Li. Axiom-based grad-cam: Towards accurate visualization and explanation of cnns. *arXiv preprint arXiv:2008.02312*, 2020.

[4] Edmund F Goh, Zhe Chen, and Wei Xian Lim. Frequency domain convolutional neural network for large image classification. *Neural Computing and Applications*, 34:19799–19812, 2022. 1, 2

[5] Sumaiya Iftikhar, Naveed Anjum, Ali B Siddiqui, Muhammad U Rehman, and Naeem Ramzan. An interpretable and simplified deep learning model for brain tumor detection using explainable ai. *IEEE Access*, 11:41003–41019, 2023. 1, 2, 3

[6] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 4765–4774, 2017. 3

[7] Jose A Marmolejo-Saucedo and Utku Kose. Numerical grad-cam based explainable convolutional neural network for brain tumor diagnosis. *Mobile Networks and Applications*, pages 1–10, 2020. 1

[8] Michael Mathieu, Mikael Henaff, and Yann LeCun. Fast training of convolutional networks through ffts. In *International Conference on Learning Representations (ICLR)*, 2014.

[9] Houda Moujahid, Brahim Cherradi, Mohammed Al-Sarem, Lahcen Bahatti, Abdel Eljialy, Abdulrahman Alsaeedi, and Fahad Saeed. Combining cnn and grad-cam for covid-19 disease prediction and visual explanation. *Intelligent Automation and Soft Computing*, 32(2):723–745, 2022.

[10] Abdullah-Al Nahid and Yinan Kong. Histopathological breast-image classification using local and frequency domains by convolutional neural network. *Information*, 9(1): 19, 2018. 2

[11] Zacharias Papanastasopoulos, Ravi K Samala, Heang-Ping Chan, Lubomir Hadjiiski, Chintana Paramagul, Mark A Helvie, and Carla H Neal. Explainable ai for medical imaging: Deep-learning cnn ensemble for classification of estrogen receptor status from breast mri. In *Medical Imaging 2020: Computer-Aided Diagnosis*, pages 356–363. SPIE, 2020. 1

[12] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. Why should i trust you? explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144, 2016. 3

[13] Nils Sassali. Deep learning with fourier transformed images. Master's thesis, KTH Royal Institute of Technology, 2022. 1, 4

[14] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, 2017. 1, 2, 3

[15] Xianlun Tang, Jiangping Peng, Bing Zhong, Jie Li, and Zhenfu Yan. Introducing frequency representation into convolution neural networks for medical image segmentation via twin-kernel fourier convolution. *Computer Methods and Programs in Biomedicine*, 205:106110, 2021. 2

[16] Simon Totterstrom. Image classification in the frequency domain with neural networks. Master's thesis, KTH Royal Institute of Technology, 2021. 1, 2, 4

[17] Nicolas Vasilache, Jeff Johnson, Michael Mathieu, Soumith Chintala, Serkan Piantino, and Yann LeCun. Fast convolutional nets with fbfft: A gpu performance evaluation. *arXiv preprint arXiv:1412.7580*, 2015.

[18] Savita Walia, Krishan Kumar, Saurabh Agarwal, and Hyunsung Kim. Using xai for deep learning-based image manipulation detection with shapley additive explanation. *Symmetry*, 14(8):1611, 2022.

[19] Haofan Wang, Rakshit Naidu, Joy Michael, and Soumya Snigdha Kundu. Ss-cam: Smoothed score-cam for sharper visual feature localization. *arXiv preprint arXiv:2006.14255*, 2020.

[20] Haofan Wang, Zifan Wang, Mengnan Du, Fan Yang, Zijian Zhang, Sirui Ding, Piotr Mardziel, and Xia Hu. Score-cam: Score-weighted visual explanations for convolutional neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 24–25, 2020.

[21] Kai Xu, Ming Qin, Fei Sun, Yu Wang, Yu-Kun Chen, and Feng Ren. Learning in the frequency domain. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1740–1749, 2020. 1, 2